IROP: IT Resource & Operations Platform

Internship Project Presentation & Interview Defense Guide

Prepared for: Tech Avenue Internship Week 1 Project Defense
Project: IT Resource & Operations Platform (IROP)
Author: Abdul Basir
Stack: MongoDB, Express.js 5, React 18, Node.js 20, TypeScript, Docker, GitHub Actions, Vercel

---

≡ƒôæ Table of Contents

1. The 3-Minute Interview Pitch
2. What is IROP & What Problem Does It Solve?
3. The Three Core Operational Modules
    - 3.1 Leave Management System (LMS)
    - 3.2 Asset Management System (AMS)
    - 3.3 Inventory Management System (IMS)
4. Role-Based Access Control (RBAC) & Hierarchy
5. Architecture & Why We Chose the MERN Stack
6. DevOps, Docker & CI/CD Pipelines
7. Cloud Deployment Strategy (Vercel & Atlas)
8. Top 15 Technical Interview Questions & Answers
9. Step-by-Step Live Demo Script

---

1. The 3-Minute Interview Pitch

> *"When the interviewer says: 'Walk me through what you built for Week 1', say this with confidence:"*

"Good morning/afternoon! For my Week 1 internship task, I built **IROP**, which stands for **IT Resource & Operations Platform**.

In many IT companies, internal operations are messy: HR tracks leaves in spreadsheets, IT records company laptops in separate documents, and office supplies are handled ad-hoc without alerts.

I designed and developed **IROP** as a **single, unified full-stack platform** combining three essential operational pillars:

1. **A Leave Management System (LMS)** with automatic weekday calculation, quota tracking, and strict managerial approval workflows.
2. **An Asset Management System (AMS)** to manage hardware laptops and software license keys across their lifecycle.

3. **An Inventory Management System (IMS)** with low-stock alerts, zero-floor protection, and quick stock adjustments.

I built this using **TypeScript across the entire MERN stack** (MongoDB Atlas, Express 5, React 18, and Node 20), secured with **JWT and bcrypt**, containerized it with **Docker**, built an automated **GitHub Actions CI/CD pipeline** that runs automated tests and publishes images to **GitHub Container Registry (GHCR)**, and deployed the production application to **Vercel**.

I also completed comprehensive architectural **R&D documentation** covering all three systems and modern container deployment. All 25 automated backend tests and frontend tests pass with 100% success."

---

2. What is IROP & What Problem Does It Solve?

The Core Problem

Most growing technology firms face **operational fragmentation**:

Operational Area	Old Broken Way	The IROP Solution
Employee Leave	Email threads, WhatsApp messages, lost quota counts.	Central portal: auto-computes weekdays, prevents overlapping dates, manager approval queue, atomic quota deduction.
IT Equipment	Stale Excel sheets; lost laptops; unassigned software keys.	Unified asset catalog: tracks hardware serials and software keys; locks broken items from assignment.
Office Inventory	Running out of chargers/cables unexpectedly; over-ordering.	Dynamic low-stock threshold badges (`quantity <= threshold`) and instant one-click adjustments.

---

3. The Three Core Operational Modules

```
+-------------------------------------------------------------------------------+ |
```

3.1 Leave Management System (LMS)

- **4 Quota Categories**: Casual, Sick, Annual, and Work-From-Home (WFH).
- **Smart Weekday Calculator**: When an employee requests leave from Thursday to Monday, the system automatically skips Saturday and Sunday, recording exactly 3 working days instead of 5 calendar days.
- **Overlap Protection**: The backend checks the database to prevent an employee from booking overlapping leaves.
- **Atomic Quota Deduction**: Quotas are **never** deducted when an employee applies or cancels. They are deducted **only when approved** by their direct manager using MongoDB's atomic `$inc` operator.

3.2 Asset Management System (AMS)

- **Polymorphic Tracking**: Tracks both physical hardware (MacBook, Dell Monitor with serial numbers) and digital licenses (JetBrains, GitHub Enterprise with digital keys).
- **Strict State Lifecycle**:

```
[ Available ] <====== Unassign ======= [ In Use ]       |                                |    Damaged
```

- **Repair Safeguard**: If an asset is marked `In Repair`, the system **strictly disables the assign button**. Broken equipment cannot be handed to employees.

3.3 Inventory Management System (IMS)

- **Consumables Tracking**: Office stationery, cables, chargers, adapters.
- **Visual Low-Stock Indicator**: If `quantity <= threshold`, a red **"Low Stock"** badge appears automatically.
- **One-Click Quick Controls**: Admin users can adjust stock on the fly with `+` and `ΓêÆ` buttons.
- **Zero-Floor Rule**: Stock quantity can never go below 0 (enforced at both API and MongoDB schema levels).

---

4. Role-Based Access Control (RBAC) & Hierarchy

IROP enforces 3 distinct roles with strict server-side validation:

```
                +----------------------------+                |     ADMIN (Ayesha Khan)     |
```

Key Security Guardrails:

1. **No Self-Approval**: A manager or admin **cannot approve their own leave request** (`403 Forbidden`).
2. **Direct-Reports Only**: Manager Bilal cannot approve leave for Sara's team (Maryam or Daniyal).
3. **Admin Demotion Guard**: The system prevents deactivating or demoting the last active administrator.

---

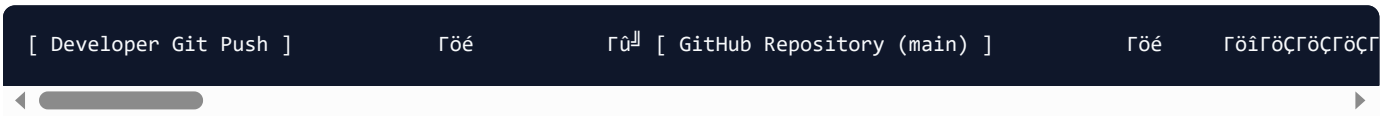
5. Architecture & Why We Chose the MERN Stack

During the interview, explaining **why** you chose this tech stack demonstrates senior engineering maturity:

Technology	Role	Why It is the Best Choice for IROP
MongoDB Atlas	Database	**Document Model**: Quotas (`casual` , `sick` , `annual`) and organizational hierarchy are stored nested inside the user document for fast single-query lookups without SQL joins. **Atomic** `$inc` : Prevents race conditions during simultaneous leave approvals.
Express.js 5 & Node.js 20	Backend API	**Asynchronous Throughput**: Event-driven I/O handles peak operational traffic with tiny server memory footprint. **Middleware**: Clean chaining of Helmet, CORS, Rate Limiting, and JWT verification.
React 18 & Vite	Frontend UI	**Instant Reactivity**: Searching through assets, calculating working days, and adjusting inventory counters happen instantaneously with zero page reloads. Fast Vite HMR and build optimization.
TypeScript (Strict)	Full-Stack Language	Eliminates language context-switching between frontend and backend. Both sides share Zod schemas and API contracts, wiping out runtime `null` errors.
TanStack Query	Data Fetching	Handles automatic caching, background refetching, and optimistic UI mutations.
Tailwind CSS	Styling	Utility-first, responsive, lightweight CSS without runtime overhead.

---

6. DevOps, Docker & CI/CD Pipelines



1. Multi-Stage Docker Strategy

- **Non-Root Security**: The backend runs under unprivileged `USER node` (UID 1001), and the frontend runs under `nginx-unprivileged` (UID 101). Even if breached, the attacker has no root access to the host.
- **Lightweight Alpine Images**: Kept images tiny (~50 MB for Node, ~25 MB for Nginx).

2. GitHub Actions CI/CD

- **Automated CI Gatekeeper**: On every push to `main` , GitHub runs the automated test suites against a containerized MongoDB service. If a test fails, the commit is flagged red.
- **Continuous Delivery (GHCR)**: Automatically builds and publishes ready-to-run container images to GitHub Packages (`ghcr.io/abdulbasir21/irop-server` and `irop-client`).

---

7. Cloud Deployment Strategy (Vercel & Atlas)

- **Frontend**: Hosted on **Vercel Edge CDN** (`irop-frontend.vercel.app`).
  - Features `client/vercel.json` SPA rewrites so refreshing subroutes (`/leave` , `/assets` , `/inventory`) never gives a `404 Not Found` .
- **Backend API**: Hosted on **Vercel Serverless Functions** (`serverirop-backend.vercel.app`).
  - Powered by `server/api/index.js` which caches the Mongoose database connection across serverless invocations.
- **Database**: Hosted on **MongoDB Atlas** (cloud cluster in AWS with automated daily backups).

8. Top 15 Technical Interview Questions & Answers

Q1: What is IROP in one sentence?

> ***Answer:*** *"IROP stands for IT Resource & Operations Platform. It is a full-stack internal enterprise portal that combines HR leave tracking, IT hardware/software asset management, and office inventory supply control in a single role-based web application."*

Q2: Why did you choose MongoDB instead of a relational database like PostgreSQL or MySQL?

> ***Answer:*** *"Two main reasons: First, IT assets are polymorphic. Hardware items have physical attributes like MAC addresses and serial numbers, while software licenses have digital keys and seat counts. In MongoDB, both exist cleanly in one collection without messy foreign keys or null columns. Second, an employee's leave quotas (`casual`, `sick`, `annual`, `wfh`) are stored as nested objects within their user document, allowing us to retrieve the user and all their balances in a single high-speed query without multi-table SQL joins."*

Q3: How do you prevent two employees from booking conflicting leaves or an employee from overlapping their own dates?

> ***Answer:*** *"We enforce date validation at the API service layer. Before saving a leave request, the server executes a MongoDB query checking if any `pending` or `approved` requests exist for that user where `startDate <= newEndDate` and `endDate >= newStartDate`. If an overlap is detected, the server immediately returns an HTTP 400 Bad Request error."*

Q4: How do you prevent race conditions when deducting leave quotas?

> ***Answer:*** *"We use MongoDB's atomic `$inc` operator. Quotas are only deducted upon approval, and the update is executed as `{ $inc: { 'quotas.annual.used': daysCount } }` alongside setting the leave status to `approved`. Because `$inc` is atomic at the document level, simultaneous approvals can never corrupt or double-deduct the balance."*

Q5: Can an Administrator or Manager approve their own leave?

> ***Answer:*** *"No. In `Leave.service.ts`, we explicitly enforce a self-approval guard: `if (Leave.userId.toString() === reviewerId) throw 403 Forbidden`. A manager's leave request must be approved by an Admin."*

Q6: Can a Manager approve leave for employees outside their team?

> ***Answer:*** *"No. When a manager attempts an approval, the server queries the database to confirm that the employee's `managerId` matches the reviewer's ID. If an engineering manager tries to approve a QA engineer's leave, the API blocks it with an HTTP 403 Forbidden."*

Q7: How does your Asset Management module handle broken hardware?

> ***Answer:*** *"Assets have a strict 3-state lifecycle: `available`, `in_use`, and `in_repair`. If an asset is marked `in_repair`, our API and frontend strictly disable assignment. It must first be repaired and transitioned back to `available` before it can be assigned to an employee."*

Q8: How does your Inventory module ensure stock never drops below zero?

> ***Answer:*** *"We enforce defense-in-depth on two levels: First, in the Mongoose schema, we define `min: [0, 'Quantity cannot be negative']`. Second, in our decrement API endpoint, we check `if (item.quantity <= 0) throw 400 Validation Error`. This guarantees stock counts never become negative."*

Q9: How is authentication and authorization handled?

> ***Answer:*** *"We use stateless JWT authentication signed with HMAC-SHA256, with passwords salted and hashed using `bcryptjs` (cost factor 10). On protected routes, our `requireAuth` middleware validates the Bearer token, and our `requireRole` middleware checks role permissions (Employee, Manager, Admin). Authorization is always enforced on the backend, never trusted from the client."*

Q10: Why did you use Zod for validation?

> ***Answer:*** *"Zod gives us runtime validation and static TypeScript type inference simultaneously. Every request body (logins, leave forms, asset creation) passes through a Zod validator middleware before reaching our business logic. Malformed payloads are rejected immediately with 400 errors, preventing invalid data from touching our database."*

Q11: What does your GitHub Actions CI pipeline do?

> ***Answer:*** *"On every git push to `main`, GitHub spins up an Ubuntu cloud virtual machine and starts a containerized MongoDB service. It runs ESLint, strict TypeScript type checking (`tsc --noEmit`), builds both frontend and backend,*

> *and executes all 25 automated backend tests and frontend tests. If any check fails, the commit is rejected with a red badge."*

Q12: What is the purpose of `docker-publish.yml`?

> ***Answer:*** *"It is our Continuous Delivery workflow. It builds production-ready, multi-stage Docker images for both the client and server and automatically publishes them to GitHub Container Registry (`ghcr.io`). This allows anyone to run our entire application using standard Docker without having Node or dependencies installed."*

Q13: What security measures were taken for Docker containers?

> ***Answer:*** *"We applied the principle of least privilege. In our Dockerfiles, neither the server nor the client runs as `root`. The server runs as unprivileged `USER node` (UID 1001), and the frontend runs on `nginx-unprivileged` (UID 101). Furthermore, we use `.dockerignore` to ensure `.env` secrets and git history are never baked into container layers."*

Q14: How did you solve CORS and deployment issues on Vercel?

> ***Answer:*** *"Two adjustments: First, on the client, we sanitized `VITE_API_URL` to strip trailing slashes, preventing double-slash `//api` redirects that browsers block during CORS preflight checks. Second, on the server, we configured CORS middleware to accept requests from all dynamic `*.vercel.app` domains and allowed preflight `OPTIONS` requests."*

Q15: If you had another week for Week 2, what would you add?

> ***Answer:*** *"I would add:*

1. Automated email notifications using Nodemailer/SendGrid when a leave request is submitted or approved.
2. File attachments (e.g., medical certificates for sick leaves) using AWS S3.
3. CSV/PDF export for asset audit reports and quarterly inventory stock audits."

---

9. Step-by-Step Live Demo Script

When demonstrating the live app to the interviewer, follow this quick 4-step sequence:

Step 1: Log in as Employee (`usman@irop.demo` / `Demo@1234`)

- Show the **Leave Dashboard**: Point out the 4 quota balance cards (Casual, Sick, Annual, WFH).

- Click **"Request Leave"**: Pick a Thursday-to-Monday date range. Point out how the dynamic calculator counts **3 working days** instead of 5 days (skipping Saturday and Sunday). Submit it.
- Show **Assets**: Point out that Usman only sees his own assigned MacBook Pro and Dell monitor.

Step 2: Log in as Manager (`bilal@irop.demo` / `Demo@1234`)

- Go to **Leave Approvals**: Show Usman's pending leave request under Bilal's team queue.
- Click **"Approve"**: Add an optional note. Point out that the status turns green.
- Show **Assets & Inventory**: Show that Bilal has read-only visibility (he cannot delete items or change stock counts).

Step 3: Log in as Admin (`admin@irop.demo` / `Demo@1234`)

- Go to **Inventory**:
  - Point out the red **"Low Stock"** badge on items below threshold.
  - Click the `+` and `-` buttons to show live stock adjustments.
  - Decrement an item to 0 to demonstrate the zero-floor protection.
- Go to **Assets**:
  - Show the search bar: type "MacBook" to show debounced filtering.
  - Show how an asset marked `In Repair` has its Assign button safely disabled.

Step 4: Show the GitHub & DevOps Pipeline

- Open your GitHub repository: **github.com/abdulbasir21/Tech-Avenue-Intership**.
- Click the **Actions** tab: Show the ≡ƒÆ ó **green checkmarks** on both the **CI Pipeline** and **Docker Publish to GHCR**.
- Show the **Packages** section on the right sidebar showing `irop-server` and `irop-client` container packages.